A nerdy kid's diary

Discovering the possibilities of Machine Learning within Structural Engineering

Simon Mjelde Løvdal
NTNU – Norwegian University of Science and Technology
01.06.2026 - 10.07. 2026

Contents

List of Figures

List of Tables

1 preliminaries

A short comparison note explaining tensors, automatic differentiation, model definition, training loops and deployment differences.

1.1 Tensors

Tensors are very similar to arrays and matrices (PyTorch, 2021). Thus, they describe a multi-linear relationship between sets of algebraic objects associated with a given vector space. Also generalizing scalars, vectors and matrices to higher dimensions (Wikipedia contributors, 2026). An important distinction is that all tensors are immutable, so you can never update the contents of a tensor. You can only create a new one (TensorFlow, 2024).

Tensors also have the ability to run on GPU's or other hardware accelerators. By default, the tensors are created on the CPU, so we explicitly have to move the tensors. Moving large tensors across devices can be expensive in terms of time and memory (PyTorch, 2021).

1.2 Automatic Differentiation

There are several methods to compute the derivatives. Like manual, numerical and automatic differentiation, just to name a few. Manual differentiation is inefficient and prone to error. Numerical differentiation is 'simple', but can introduce large round-off and truncation errors (Baydin et al., 2018, p. 2).

Automatic differentiation computes the derivative through accumulation of values during execution, rather than derivative expressions. This allows for a precise evaluation of derivatives with machine precision (Baydin et al., 2018, p. 2).

For further reading take a look at Baydin et al. (Baydin et al., 2018) or Andrew Holmes (Holmes, 2024).

1.3 Model Definition

PyTorch and TensorFlow are both deep learning frameworks, and both provide the necessary tools for defining, training and evaluating. Both also support GPU acceleration.

TensorFlow is preferred for high-level production workflows. Keras' API makes it easy to create simple models. Which for example is useful for a sequence of dense layers.

PyTorch is preferred as it closely follow the normal Python programming style. A PyTorch model will also give more flexibility and make the internal operations easier to inspect.

1.3.1 Comparison

```python
import torch
import torch.nn as nn

class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer = nn.Linear(10, 1)

    def forward(self, x):
        return self.layer(x)
```

Listing 1: Simple PyTorch model

```python
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='relu', input_shape=(10,),
    tf.keras.layers.Dense(1)
])
```

Listing 2: Simple TensorFlow model

1.3.2 PyTorch Training Run

Training is also a little bit different for PyTorch. You have to write it out manually, which means creating a forward pass, loss calculation, gradient reset, backpropagation and optimizer update.. Which does sound like a mouthful, but once quickly get used to it.

```python
for x, y in dataloader:
    optimizer.zero_grad()
    prediction = model(x)
    loss = loss_fn(prediction, y)
    loss.backward()
    optimizer.step()
```

Listing 3: Training run using PyTorch

1.4 Deployment Differences

This might not be a surprise, but the deployment between the two (even though similar) have their differences. But this is mainly down to parameter settings. This is just something

we need to take into consideration when comparing the two. Hyper parameter tuning is something we will look into at a later point.

References

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, *18*(153), 1–43. https://arxiv.org/abs/1502.05767

Holmes, A. (2024). What's automatic differentiation? [2026-06-03]. https://huggingface.co/blog/andmholm/what-is-automatic-differentiation

PyTorch. (2021, February). *Tensors*. Retrieved June 3, 2026, from https://docs.pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html

TensorFlow. (2024, August). *Introduction to Tensors*. Retrieved June 3, 2026, from https://www.tensorflow.org/guide/tensor

Wikipedia contributors. (2026, June). *Tensor*. Retrieved June 3, 2026, from https://en.wikipedia.org/wiki/Tensor